



Presenter Guide — Session 3: Save It For Real

Duration: 90 minutes · **Outcome:** a persistent SQLite CRUD app that survives restarts, plus the final-project kickoff.

At a glance

Block	Time	Topic
1	0:00–0:05	Recap the data-loss pain · today's goal (show finished app)
2	0:05–0:11	Memory vs storage · SQLite · CRUD ↔ SQL
3	0:11–0:19	Install/open <code>expo-sqlite</code> · <code>CREATE TABLE</code> · types
4	0:19–0:27	INSERT / SELECT / UPDATE / DELETE · <code>?</code> placeholders
5	0:27–0:34	<code>db.js</code> module · <code>useEffect</code> load · refresh pattern
6	0:34–0:40	Modal (add+edit) · <code>Alert</code> confirm · live-code
7	0:40–1:10	Activity 3: make tasks permanent
8	1:10–1:30	Final project kickoff · course recap · send-off



Protect the last 18 minutes for the final-project kickoff — it's the whole point of the workshop's payoff. If you're behind, trim the `?`-placeholder deep-dive (keep the one-line "always use `?`") and the navigation slide.

Before class

- ☐ `git checkout session-3 && npm install && npx expo start` runs the finished CRUD app.
- ☐ Practice the **persistence demo**: add tasks → **force-close** (swipe away) → reopen → still there.
- ☐ Skim `final-project/BRIEF.md` so you can speak to options + rubric.

Block-by-block

Blocks 1–2 — Motivation + concepts (0:00–0:11)

- Re-demo data loss on the session-2 app (force-close → gone). Feel the pain.
- **Memory vs storage**: state = live picture; database = permanent record. They work together.

 **ELABORATE:** SQLite is the most-deployed DB in the world; it lives in a file *inside the app*, no server/internet. "SQL" = the language. **CRUD = Create/Read/Update/Delete = INSERT/SELECT/UPDATE/DELETE**. Almost every app is CRUD on some data — which means after today they can build almost anything.

Block 3 — Setup + table (0:11–0:19)

 **ELABORATE:** `npx expo install expo-sqlite` (not plain npm) matches the SDK version. `openDatabaseSync('tasks.db')` opens/creates the file. Walk the schema: `id INTEGER PRIMARY KEY` (auto-numbered unique id), `title TEXT`, `done INTEGER DEFAULT 0` (SQLite has no boolean → 0/1). `IF NOT EXISTS` makes it safe to run on every launch.

- Types to teach: `INTEGER`, `TEXT`, `REAL` only. No BOOLEAN/DATE.

Block 4 — The four operations (0:19–0:27)

 **ELABORATE:** `runSync` for changes (INSERT/UPDATE/DELETE), returns `{ lastInsertRowId, changes }`. `getAllSync` for reads, returns an **array of row objects** — drops straight into `FlatList`. The `?` placeholders are filled by the params in order.

-  **Hammer the `WHERE id = ?` warning** — UPDATE/DELETE without WHERE hits every row.
-  **Always `?`, never glue strings** — apostrophes break it, and string-gluing is SQL injection.

Block 5 — Module + wiring (0:27–0:34)

 **ELABORATE:** Put all SQL in `db.js` ; App.js calls friendly functions. `useEffect(() => {...}, [])` runs **once on startup**: create table, then load. **The golden pattern:** after *any* change, call `refresh()` to re-read the DB into state. DB = source of truth; state = snapshot for the screen. Simple and bug-resistant for beginners.

Block 6 — Modal + Alert (0:34–0:40)

 **ELABORATE:** A `Modal` is a pop-up "second screen". One form does add **and** edit: `editingId === null` → add, else → update that id. `Alert.alert(...)` gives a native confirm dialog before deleting. Mention Expo Router / React Navigation as the real multi-screen tools (stretch / final project).

- **Live-code** `db.js` then `App.js` . End by force-closing & reopening to prove persistence.
-

✓ Full solution — `db.js` (matches `session-3` branch)

```
import * as SQLite from 'expo-sqlite';

// open (or create) the database file on the device
const db = SQLite.openDatabaseSync('tasks.db');

// create the table once (safe to call on every launch)
export function setupDatabase() {
  db.execSync(`
    CREATE TABLE IF NOT EXISTS tasks (
      id    INTEGER PRIMARY KEY NOT NULL,
      title TEXT    NOT NULL,
      done  INTEGER NOT NULL DEFAULT 0
    );
  `);
}

// READ – newest first
export function getTasks() {
  return db.getAllSync('SELECT * FROM tasks ORDER BY id DESC');
}

// CREATE
export function addTask(title) {
  const result = db.runSync('INSERT INTO tasks (title, done) VALUES (?, ?)', title, 0);
  return result.lastInsertRowId;
}

// UPDATE (title)
export function updateTask(id, title) {
  db.runSync('UPDATE tasks SET title = ? WHERE id = ?', title, id);
}

// UPDATE (done 0/1)
export function toggleTask(id, done) {
  db.runSync('UPDATE tasks SET done = ? WHERE id = ?', done ? 1 : 0, id);
}

// DELETE
export function deleteTask(id) {
  db.runSync('DELETE FROM tasks WHERE id = ?', id);
}
```

✓ Full solution — App.js (matches session-3 branch)

```
import { StatusBar } from 'expo-status-bar';
import { useEffect, useState } from 'react';
import {
  Alert, FlatList, Modal, StyleSheet, Text, TextInput, TouchableOpacity, View,
} from 'react-native';
import * as DB from './db';

function TaskRow({ task, onToggle, onEdit, onDelete }) {
  return (
    <View style={styles.item}>
      <TouchableOpacity style={styles.itemLeft} onPress={onToggle}>
        <Text style={styles.checkbox}>{task.done ? '✓' : '□'}</Text>
        <Text style={[styles.itemText, task.done && styles.itemTextDone]}>{task.title}</Text>
      </TouchableOpacity>
      <TouchableOpacity onPress={onEdit} hitSlop={8}><Text style={styles.action}>✎</Text>
      <TouchableOpacity onPress={onDelete} hitSlop={8}><Text style={styles.action}>🗑️</Text>
    </View>
  );
}

export default function App() {
  const [tasks, setTasks] = useState([]);
  const [modalVisible, setModalVisible] = useState(false);
  const [draft, setDraft] = useState('');
  const [editingId, setEditingId] = useState(null); // null = adding

  useEffect(() => {
    DB.setupDatabase(); // create the table on first run
    refresh(); // load whatever is already saved
  }, []);

  function refresh() {
    setTasks(DB.getTasks()); // re-read the DB into state
  }

  function openAdd() {
    setEditingId(null); setDraft(''); setModalVisible(true);
  }

  function openEdit(task) {
    setEditingId(task.id); setDraft(task.title); setModalVisible(true);
  }

  function save() {
    const title = draft.trim();
    if (title === '') return;
    if (editingId === null) DB.addTask(title); // CREATE
    else DB.updateTask(editingId, title); // UPDATE
  }
}
```

```

    setModalVisible(false);
    refresh();
  }
  function toggle(task) {
    DB.toggleTask(task.id, !task.done);          // UPDATE done
    refresh();
  }
  function remove(task) {
    Alert.alert('Delete task?', ` "${task.title}"`, [
      { text: 'Cancel', style: 'cancel' },
      { text: 'Delete', style: 'destructive',
        onPress: () => { DB.deleteTask(task.id); refresh(); } },    // DELETE
    ]);
  }

  const remaining = tasks.filter((t) => !t.done).length;

  return (
    <View style={styles.screen}>
      <StatusBar style="light" />

      <View style={styles.header}>
        <Text style={styles.title}>My Tasks</Text>
        <Text style={styles.subtitle}>{remaining} left · saved on this device 📁</Text>
      </View>

      <FlatList
        data={tasks}
        keyExtractor={(item) => String(item.id)}
        contentContainerStyle={styles.list}
        ListEmptyComponent={<Text style={styles.empty}>No tasks yet. Tap + to add one! ✨</Text>}
        renderItem={({ item }) => (
          <TaskRow task={item}
            onToggle={() => toggle(item)} onEdit={() => openEdit(item)} onDelete={() => r
          )}
        >
      </FlatList>

      <TouchableOpacity style={styles.fab} onPress={openAdd}>
        <Text style={styles.fabText}>+</Text>
      </TouchableOpacity>

      <Modal visible={modalVisible} animationType="slide" transparent>
        <View style={styles.modalBackdrop}>
          <View style={styles.modalCard}>
            <Text style={styles.modalTitle}>{editingId === null ? 'New Task' : 'Edit Task'}</Text>
            <TextInput
              style={styles.modalInput} placeholder="What needs doing?" placeholderTextColor={
                'gray'} value={draft} onChangeText={setDraft} autoFocus onSubmitEditing={save} />
            <View style={styles.modalButtons}>
              <TouchableOpacity style={[styles.modalBtn, styles.cancelBtn]} onPress={() => {

```

```

        <Text style={styles.cancelText}>Cancel</Text>
      </TouchableOpacity>
      <TouchableOpacity style={[styles.modalBtn, styles.saveBtn]} onPress={save}>
        <Text style={styles.saveText}>Save</Text>
      </TouchableOpacity>
    </View>
  </View>
</View>
</Modal>
</View>
);
}

const styles = StyleSheet.create({
  screen: { flex: 1, backgroundColor: '#F1F5F9', paddingTop: 60 },
  header: { paddingHorizontal: 20, paddingBottom: 12 },
  title: { fontSize: 32, fontWeight: 'bold', color: '#0F172A' },
  subtitle: { fontSize: 14, color: '#64748B', marginTop: 2 },
  list: { paddingHorizontal: 20, paddingBottom: 100 },
  item: { flexDirection: 'row', alignItems: 'center', backgroundColor: 'FFFFFF', borderR
  itemLeft: { flexDirection: 'row', alignItems: 'center', flex: 1, gap: 12 },
  checkbox: { fontSize: 20 },
  itemText: { fontSize: 16, color: '#0F172A', flexShrink: 1 },
  itemTextDone: { textDecorationLine: 'line-through', color: '#94A3B8' },
  action: { fontSize: 18 },
  empty: { textAlign: 'center', color: '#94A3B8', marginTop: 60, fontSize: 16 },
  fab: {
    position: 'absolute', right: 24, bottom: 36, width: 60, height: 60, borderRadius: 30,
    backgroundColor: '#4338CA', alignItems: 'center', justifyContent: 'center',
    elevation: 6, shadowColor: '000', shadowOpacity: 0.3, shadowRadius: 8, shadowOffset:
  },
  fabText: { color: 'FFFFFF', fontSize: 32, lineHeight: 34 },
  modalBackdrop: { flex: 1, backgroundColor: 'rgba(15,23,42,0.5)', justifyContent: 'flex-
  modalCard: { backgroundColor: 'FFFFFF', borderTopLeftRadius: 24, borderTopRightRadius:
  modalTitle: { fontSize: 20, fontWeight: 'bold', color: '#0F172A', marginBottom: 16 },
  modalInput: { backgroundColor: '#F1F5F9', borderRadius: 12, paddingHorizontal: 16, padd
  modalButtons: { flexDirection: 'row', gap: 12, marginTop: 16 },
  modalBtn: { flex: 1, borderRadius: 12, paddingVertical: 14, alignItems: 'center' },
  cancelBtn: { backgroundColor: '#F1F5F9' },
  cancelText: { color: '#475569', fontWeight: '600', fontSize: 16 },
  saveBtn: { backgroundColor: '#4338CA' },
  saveText: { color: 'FFFFFF', fontWeight: '600', fontSize: 16 },
});

```

Block 7 — Activity 3 facilitation (0:40–1:10)

MVP for everyone: Create + Read + Delete that *persists* across a restart. Update/edit-modal is the stretch.

Common student errors & fixes

Symptom	Cause	Fix
Cannot find module 'expo-sqlite'	Not installed	<code>npm install expo-sqlite</code> , restart the server
List empty after adding	Forgot to <code>refresh()</code> after the change	Call <code>refresh()</code> right after every DB write
Nothing loads on open	Forgot <code>useEffect</code> / <code>setupDatabase</code>	<pre>useEffect(() => { DB.setupDatabase(); refresh(); }, [])</pre>
Changed/deleted everything	Missing <code>WHERE id = ?</code>	Always target one row by id
<code>done</code> shows weird / won't strike	It's <code>0 / 1</code> , not true/false	Treat <code>task.done</code> as truthy (1) / falsy (0)
Old data won't go away after schema change	Table already created	Uninstall the app from the phone (drops the DB file) and reopen
App crashes on a quote in the title	Glued the string instead of using <code>?</code>	Use placeholders: <code>VALUES (?)</code> + param

 **Facilitation:** The #1 fix is "did you `refresh()` after the change?" The #2 is the missing `WHERE`. Keep the persistence demo as the carrot — getting data to survive a restart is the moment that makes it all click.

Block 8 — Final project + send-off (1:10–1:30)

- Walk `final-project/BRIEF.md`: options, MVP requirements, stretch, rubric, timeline (suggest 1–2 weeks + a demo day).
- Reassure: "the final project is the **same patterns** on **different data**. You already have every skill."

- **Course-wide recall:** components? state? CRUD letters? Celebrate the journey from zero.
- Share how to ask questions / submit. Encourage starting within a week while it's fresh. End on time.